

Week 5 · 개념 워크북: 코드를 재사용 단위로 묶고, 모듈끼리 값을 넘깁니다

참고

이 문서는 개념 파트입니다. 손으로 치는 실습은 같은 `lecture/` 폴더의 **실습워크북** 을 따라가세요. [개념] 은 개념 설명, [함정] 은 미리 알고 피해 가는 것입니다.

지난주에 값을 코드 밖으로 꺼내고 `for_each` 로 서브넷을 두 개 만들었습니다. 그런데 그 코드는 폴더 하나에 평평하게 놓여 있습니다. 오늘 그것을 `modules/network` 와 `modules/compute` 로 나누고 `envs/prod` 에서 조립합니다.

이 문서를 덮을 때 세 가지를 말할 수 있어야 합니다. 모듈의 경계를 무엇으로 긋는지, 모듈 사이로 값이 어떻게 건너가는지, 모듈로 옮기면 state 주소가 어떻게 바뀌는지입니다.

! 꼭 기억

week1·2·3·4에서 이미 설명한 것은 다시 풀지 않고 위치만 가리킵니다. VPC · 서브넷 · IGW · 라우트 테이블 · 시큐리티 그룹 · AMI · CIDR 은 week2 개념워크북 Part 0 · Part 1 · Part 2에 있고, backend · 원격 state · 잠금은 week3 개념워크북 Part 0에, `variable` · `locals` · `merge()` · `for_each` · `validation` · `sensitive` 는 week4 개념워크북에 있습니다.

오늘은 week4 개념워크북 19번이 Week5로 넘긴 표를 그대로 회수합니다.

week4에서 넘긴 것	오늘 어디서 받는가
<code>variables.tf</code> 하나에 변수가 모였습니다. 이 묶음을 다른 폴더에서도 쓰려면 파일을 복사해야 합니다	Part 1
<code>local.common_tags</code> 를 이 폴더 안에서만 씁니다. <code>locals</code> 는 폴더 경계를 넘지 않습니다	Part 2의 6번
<code>outputs.tf</code> 에서 <code>subnet_ids</code> 를 맵으로 꺼냈습니다. 코드가 코드에게 넘기려면 받는 쪽이 필요합니다	Part 3의 7 · 8번
<code>aws_subnet.public["a"]</code> 로 하나를 골랐습니다. 그 서브넷이 모듈 안으로 들어가면 밖에서 어떻게 고를까요	Part 3의 9번
<code>for_each</code> 로 서브넷을 2개로 늘렸습니다. 리소스 하나가 아니라 리소스 묶음을 여러 벌 만들려면 어디에 걸까요	Part 5의 14번
<code>backend.tf</code> 에 버킷 이름을 또 손으로 적었습니다. dev 와 prod 를 같은 코드로 만들려면 <code>key</code> 만 달라야 합니다	Part 5의 13번

오늘 다룰 것

참고

세션은 60분이고 그중 45분이 실습입니다. 그래서 이 문서는 대부분 예습용입니다. 라이브에서 짚는 것은 **Part 1·2·3·4·6**의 일부인데, 개념만 따로 떼어 설명하는 시간은 잡지 않습니다. 실습워크북 Block A 워크스루(18분) 안에서 손을 움직이면서 함께 짚습니다. 나머지는 읽고 오세요. 읽지 않으면 실습 중에 왜 폴더를 나누는지 모른 채 파일만 옮기게 됩니다.

Part	섹션	무엇을 다루나	언제
Part 0	용어 사전	오늘 처음 나오는 말 14개	예습
Part 1	1·2·3	복사본이 만드는 문제와 경계 긋기	예습·Block A 에서
Part 2	4·5·6	모듈의 안과 밖	예습·Block A 에서
Part 3	7·8·9	모듈끼리 값을 넘기기	Block A 에서
Part 4	10·11·12	주소가 길어집니다	예습·Block A 에서
Part 5	13·14	환경을 폴더로 가르기	예습
Part 6	15·16	비용과 정리	Block C 에서
Part 7	17	다음 주로 넘기는 문제	예습

오늘 나와야 하는 숫자

실습 중에 이 숫자와 다르면 멈추고 물어보세요.

지점	나와야 하는 값
<code>init</code> 모듈 초기화	두 줄. A-4 에서 <code>- network in ../../modules/network</code> , B-2 에서 <code>- compute in ../../modules/compute</code>
<code>plan</code>	<code>Plan: 9 to add, 0 to change, 0 to destroy.</code>
<code>state list</code>	10줄 (리소스 9 + 데이터 소스 1)
S3 객체	<code>week05/prod/terraform.tfstate</code> 1개
<code>destroy</code>	<code>Destroy complete! Resources: 9 destroyed.</code>

리소스 9개의 내역은 VPC 1·IGW 1·서브넷 2·라우트 테이블 1·라우트 테이블 연결 2·시큐리티 그룹 1·EC2 1입니다. 만드는 것의 종류와 개수는 week4와 같고, 달라지는 것은 그것들이 어느 모듈 안에 들어가는지입니다.

⚠ 주의

week4에서는 `state list` 가 **11줄**이었습니다. 오늘은 **10줄**입니다. 리소스 9개는 그대로인데 데이터 소스가 2개에서 1개로 줄었기 때문입니다. week4에 있던 `data.aws_availability_zones` 가 오늘 코드에는 없습니다. 서브넷 맵의 key 자체가 AZ 이름이라 AWS에 목록을 물어보지 않습니다.

이 변화가 무엇을 잃게 하는지는 Part 3의 9번과 Part 4에서 다시 봅니다.

Part 0. 용어 사전

오늘 처음 나오는 단어를 실습 전에 다 풀어둡니다. week1·2·3·4에서 정의한 용어는 넣지 않았습니다. 비유는 이 표 안에서만 쓰고, 본문에서는 왼쪽 열의 실제 이름으로 부릅니다.

용어	뜻	비유
모듈 (module)	한 폴더에 모인 <code>.tf</code> 파일 묶음. Terraform 은 파일 단위가 아니라 폴더 단위 로 코드를 읽습니다	-
루트 모듈 (root module)	<code>terraform</code> 명령을 실행하는 폴더. 오늘은 <code>envs/prod</code> 입니다. <code>provider</code> 와 <code>backend</code> 는 여기에만 둡니다	-
자식 모듈 (child module)	<code>module</code> 블록으로 불러 오는 모듈. 오늘은 <code>modules/network</code> 와 <code>modules/compute</code> 입니다	-
<code>module</code> 블록 (모듈 호출)	자식 모듈을 불러오면서 입력값을 넘기는 블록. <code>module "network" { ... }</code> 의 라벨 <code>network</code> 가 주소의 일부가 됩니다	-
<code>source</code>	모듈 코드가 어디 있는지 적는 인자. 오늘은 로컬 상대경로 <code>"../../modules/network"</code> 입니다	-
로컬 모듈 (local module)	<code>./</code> 또는 <code>../</code> 로 시작하는 <code>source</code> 가 가리키는, 같은 리포 안의 모듈	-
모듈 레지스트리 (module registry)	공개된 모듈을 받아오는 곳. <code>source</code> 가 <code>./</code> 나 <code>../</code> 로 시작하지 않으면 Terraform 이 레지스트리 주소로 해석합니다	-
입출력 계약	모듈의 <code>variables.tf</code> (받는 것)와 <code>outputs.tf</code> (내주는 것). 모듈을 쓰는 쪽에서 보이는 것은 이 둘뿐입니다	자판기의 버튼과 배출구
캡슐화 (encapsulation)	모듈 안의 리소스를 밖에서 직접 가리킬 수 없는 성질. <code>output</code> 으로 내주지 않은 것은 없는 것과 같습니다	-
provider 상속 (provider inheritance)	루트 모듈에 설정한 <code>provider</code> 가 자식 모듈로 자동으로 내려가는 것. 그래서 모듈은 <code>provider</code> 블록을 갖지 않습니다	-
모듈 경로 접두사	모듈 안 리소스의 state 주소 앞에 붙는 <code>module.<라벨>.</code> 부분. <code>module.network.aws_vpc.this</code> 의 앞 두 토막입니다	사물함 앞에 붙은 층 번호
<code>.terraform/modules</code>	<code>init</code> 이 자식 모듈의 위치를 적어두는 곳. 모듈을 추가하거나 <code>source</code> 를 바꾸면 여기가 갱신되어야 합니다	-
<code>terraform get</code>	모듈만 다시 받아오는 명령. <code>init</code> 이 이 일을 함께 하므로 따로 쓸 일은 드뭅니다	-

용어	뜻	비유
moved 블록	리소스의 주소가 바뀌었다는 것을 Terraform 에 알려서, 삭제와 생성이 아니라 이름 변경으로 처리하게 하는 블록	이사 신고

Part 1. 복사본이 만드는 문제

[개념] 1. week4 코드가 놓여 있던 모양

week4의 `practice/` 폴더에는 파일이 이렇게 놓여 있었습니다.

```
practice/
├── versions.tf · providers.tf · backend.tf
├── variables.tf · locals.tf
├── network.tf      VPC · IGW · 서브넷 · 라우트 테이블 · 연결
├── compute.tf     시큐리티 그룹 · EC2
└── outputs.tf
```

파일 이름이 `network.tf` 와 `compute.tf` 로 나뉘어 있으니 이미 나눈 것처럼 보입니다. 그런데 이것은 경계가 아닙니다. `compute.tf` 의 `aws_instance.web` 이 `network.tf` 의 `aws_subnet.public["a"]` 를 아무 절차 없이 그대로 참조했습니다. 반대 방향도 막혀 있지 않았습니다.

Terraform 은 폴더 하나를 한 덩어리로 읽습니다. 같은 폴더 안에서 파일을 몇 개로 쪼개든 변수와 리소스는 전부 같은 이름 공간에 있습니다. 파일 분리는 사람이 찾기 쉽게 한 것이고, 코드에는 아무 제약을 만들지 않습니다.

그래서 오늘 경계를 다시 긁습니다. Terraform 이 인정하는 경계는 폴더입니다.

[개념] 2. 폴더를 복사하면 조용히 어긋납니다

`prod` 말고 `dev` 도 만들어야 한다고 해봅시다. 지금 구조에서 할 수 있는 것은 폴더를 통째로 복사하는 것뿐입니다.

```
practice/      prod 용
practice-dev/  복사본
```

복사한 순간 리소스 정의가 두 벌이 됩니다. 두 벌이 되면 세 가지가 따라옵니다.

첫째, 고칠 일이 생기면 두 군데를 고쳐야 합니다. `map_public_ip_on_launch` 를 한쪽만 고쳐도 Terraform 은 오류를 내지 않습니다. 양쪽 다 문법이 맞기 때문입니다. 리소스의 동작만 조용히 달라집니다.

둘째, 어느 쪽이 원본인지 알 수 없게 됩니다. 몇 주 뒤에 두 파일을 나란히 놓고 `diff` 를 떠서 어느 차이가 의도한 것이고 어느 차이가 빼먹은 것인지 사람이 판단해야 합니다.

셋째, 세 번째 환경이 필요해지면 어느 쪽을 복사할지부터 정해야 합니다.

week4에서 값을 변수로 뺀 것은 이 문제를 절반만 풀었습니다. 값은 밖으로 나갔지만 **리소스 정의 자체가 여전히 폴더에 묶여 있습니다.**

[함정] 3. 나누는 기준은 파일이 아니라 폴더입니다

경계를 그을 때 흔히 하는 실수가 파일을 더 잘게 쪼개는 것입니다. `network.tf` 를 `vpc.tf` · `subnet.tf` · `route.tf` 로 나누면 정리된 것처럼 보이지만 재사용 관점에서는 아무것도 바뀌지 않습니다. 여전히 폴더 하나이므로 `dev` 를 만들려면 폴더를 복사해야 합니다.

오늘 만드는 구조는 이렇습니다.

```
practice/
├── modules/
│   ├── network/    VPC · IGW · 서브넷 · 라우트 테이블 · 연결
│   └── compute/    시큐리티 그룹 · EC2
└── envs/
    └── prod/        모듈 두 개를 조립합니다. provider 와 backend 가 여기 있습니다
```

`modules/network` 와 `modules/compute` 는 각각 폴더이므로 Terraform 이 서로 다른 덩어리로 봅니다. 이제 `envs/dev` 를 추가할 때 복사하는 것은 조립하는 쪽 폴더뿐이고, 리소스 정의는 한 벌로 남습니다.

무엇을 한 모듈로 묶을지는 **같이 만들어지고 같이 지워지는**가로 가릅니다. VPC 없이 서브넷만 있을 수 없고 서브넷 없이 라우트 테이블 연결만 있을 수 없으니 이 다섯은 한 모듈입니다. 시큐리티 그룹과 EC2 는 VPC 가 있어야 만들어지지만 VPC 를 지우지 않고도 지울 수 있으니 다른 모듈입니다.

Part 2. 모듈의 안과 밖

Part 1에서 경계를 폴더로 그었습니다. 이제 그 폴더를 불러오는 문법과, 경계가 생기면서 막히는 것들을 봅니다.

[개념] 4. `module` 블록과 `source`

자식 모듈은 `module` 블록으로 부릅니다. `envs/prod/main.tf` 에 이렇게 적습니다.

```
module "network" {
  source      = "../../modules/network"
  vpc_cidr    = var.vpc_cidr
  subnet_cidrs = var.subnet_cidrs
  name_prefix = local.name_prefix
  tags        = local.common_tags
}
```

`source` 를 뺀 나머지 인자는 전부 그 모듈의 `variables.tf` 에 선언된 변수 이름입니다. 모듈에 없는 이름을 적으면 오타가 그 자리에서 걸립니다.

```
Error: Unsupported argument

on main.tf line 4, in module "network":
  4:   vpc_name      = "typo"

An argument named "vpc_name" is not expected here.
```

반대로 `default` 가 없는 변수를 안 넘기면 이렇게 막힙니다. 오류가 가리키는 곳은 인자 줄이 아니고 `module` 블록의 첫 줄입니다. 어느 인자가 빠졌는지는 마지막 줄에 나오므로, 그 모듈의 `variables.tf` 와 대조하세요.

```
Error: Missing required argument

on main.tf line 1, in module "network":
  1: module "network" {

The argument "vpc_cidr" is required, but no definition was found.
```

블록 라벨 `"network"` 는 그냥 이름이 아닙니다. 이 라벨이 state 주소의 일부가 됩니다. Part 4의 10번에서 다시 봅니다.

⚠ 주의

`source` 의 로컬 경로는 `./` 또는 `../` 로 시작해야 합니다. `source = "modules/network"` 처럼 적으면 Terraform 이 이것을 로컬 경로로 보지 않고 원격 모듈 주소로 해석합니다. 폴더가 실제로 그 자리에 있어도 실패합니다.

```
Error: Invalid module source address

on main.tf line 2, in module "network":
  2:   source      = "modules/network"

Terraform failed to determine your intended installation method for remote
module package "modules/network".

If you intended this as a path relative to the current module, use
"./modules/network" instead. The "./" prefix indicates that the address is a
relative filesystem path.
```

메시지가 고칠 방법까지 알려줍니다. `./` 접두어가 "이것은 파일 경로다"라는 표시입니다.

`envs/prod` 에서 `modules/network` 까지는 두 단계를 올라가야 하므로 `../..` 입니다. `envs/prod` → `envs` → `practice` 로 올라간 다음 `modules/network` 로 내려갑니다.

[개념] 5. 모듈은 `provider` 를 갖지 않습니다

이것이 이번 주 사전 연습 확인 항목입니다.

루트 모듈에 적은 `provider` 설정은 자식 모듈로 자동으로 내려갑니다. 그래서 `modules/network/main.tf` 에 `provider` 블록이 없어도 그 안의 `aws_vpc.this` 가 서술 리전에 만들어집니다. `envs/prod/providers.tf` 한 곳이 리전을 정합니다.

```
# envs/prod/providers.tf : 루트에만 둡니다
provider "aws" {
  region = var.region
}
```

모듈 안에 `provider` 블록을 넣으면 그 모듈은 자기 리전을 스스로 정해버립니다. 그러면 이 모듈을 쓰는 사람은 리전을 바꿀 수 없고, 리전이 다른 두 환경에서 같은 모듈을 쓰는 일이 불가능해집니다. 재사용 단위로 만들려고 모듈을 뽑았는데 그 안에 환경 설정을 박아두면 뽑은 이유가 사라집니다.

📌 참고

여러 리전이나 여러 계정에 동시에 만들어야 하는 모듈은 `configuration_aliases` 로 "나는 provider 두 개를 받는다"고 선언하고, 부르는 쪽이 `providers = { ... }` 인자로 넘겨줍니다. 설정을 모듈 안에 박아두는 대신 받는 자리만 선언하므로 5번의 원칙과 어긋나지 않습니다. 오늘 실습에서는 쓰지 않습니다.

`required_providers` 도 이 실습에서는 루트의 `envs/prod/versions.tf` 한 곳에만 둡니다.

[개념] 6. 변수와 `locals` 는 폴더 경계를 넘지 않습니다

week4에서 `local.common_tags` 를 만들어 모든 리소스에 태그를 붙였습니다. 그 코드를 그대로 `modules/network/main.tf` 로 옮기면 오류가 납니다. `locals` 는 그것을 선언한 폴더 안에서만 보입니다. `var.` 도 같습니다. `envs/prod/variables.tf` 에 선언한 `var.vpc_cidr` 은 모듈 안에서 보이지 않습니다.

경계가 생겼으니 값은 문을 통해 들어가야 합니다. 그 문이 모듈의 `variables.tf` 입니다.

week4에서는	오늘 모듈 안에서는
<code>local.name_prefix</code>	<code>var.name_prefix</code> (루트가 <code>local.name_prefix</code> 를 넘겨줍니다)
<code>local.common_tags</code>	<code>var.tags</code> (루트가 <code>local.common_tags</code> 를 넘겨줍니다)
<code>var.vpc_cidr</code>	<code>var.vpc_cidr</code> (이름은 같지만 다른 변수입니다. 루트가 넘겨줍니다)

이름이 같아도 다른 변수라는 점이 헛갈립니다. `modules/network/variables.tf` 의 `vpc_cidr` 과 `envs/prod/variables.tf` 의 `vpc_cidr` 은 각자 자기 폴더의 변수이고, 둘을 잇는 것은 `main.tf` 의 `vpc_cidr = var.vpc_cidr` 한 줄뿐입니다. 왼쪽은 모듈의 변수이고 오른쪽은 루트의 변수입니다.

`name_prefix` 와 `common_tags` 를 `envs/prod/locals.tf` 에서만 정의하는 이유도 여기에 있습니다. 태그 규칙을 모듈마다 따로 두면 두 모듈이 서로 다른 태그를 붙이기 시작합니다. 규칙은 조립하는 쪽에 한 벌만 두고, 모듈은 받아서 `merge()` 로 자기 `Name` 만 엮습니다.

```
# modules/network/main.tf
tags = merge(var.tags, { Name = "${var.name_prefix}-vpc" })
```

Part 3. 모듈끼리 값을 넘기기

Part 2에서 경계를 만들고 값이 들어가는 문을 봤습니다. 이제 나오는 문을 봅니다. 오늘 라이브에서 가장 오래 짚는 부분입니다.

[개념] 7. 입출력 계약: 모듈은 세 파일로 나뉩니다

모듈 폴더 안의 파일은 역할이 셋입니다.

파일	역할	누가 씁니까
<code>variables.tf</code>	받는 것. 모듈의 입력	실습에서 이미 완성되어 제공됩니다
<code>main.tf</code>	처리. 리소스 정의	오늘 수강생이 채웁니다
<code>outputs.tf</code>	내주는 것. 모듈의 출력	실습에서 이미 완성되어 제공됩니다

`variables.tf` 와 `outputs.tf` 를 묶어 입출력 계약이라고 부릅니다. 모듈을 쓰는 쪽에서 보이는 것은 이 둘뿐이고, `main.tf` 에 리소스를 몇 개 어떻게 만들었는지는 보이지 않습니다.

실습에서 계약을 미리 주고 `main.tf` 만 채우게 하는 것은 실무 순서와 같습니다. 모듈을 만들 때 먼저 정하는 것이 "무엇을 받아서 무엇을 내줄 것인가"이고, 그 안을 어떻게 채우는지는 나중에 바꿀 수 있습니다. 계약을 바꾸면 그 모듈을 부르는 모든 곳을 고쳐야 하지만, `main.tf` 안쪽만 바꾸면 부르는 쪽은 아무것도 모릅니다.

`modules/network` 의 계약은 이렇습니다.

```
# variables.tf : 받는 것
variable "vpc_cidr" {
  description = "이 모듈이 만들 VPC 의 CIDR 블록"
  type       = string
}
variable "subnet_cidrs" {
  description = "퍼블릭 서브넷. key 가 AZ 이름, 값이 CIDR 입니다"
  type       = map(string)
}
variable "name_prefix" {
  description = "Name 태그 앞에 붙일 접두어. 루트가 넘겨줍니다"
  type       = string
}
variable "tags" {
  description = "공통 태그. 모듈은 Name 만 merge 로 엮습니다"
  type       = map(string)
  default    = {}
}

# outputs.tf : 내주는 것
output "vpc_id" {
  description = "만들어진 VPC 의 ID. compute 모듈로 넘깁니다"
  value       = aws_vpc.this.id
}
output "subnet_ids" {
  description = "AZ 이름을 key 로 하는 서브넷 ID 맵"
  value       = { for k, s in aws_subnet.public : k => s.id }
}
output "public_subnet_id" {
  description = "모듈이 대신 골라 내주는 서브넷. 개념워크북 9번"
  value       = values(aws_subnet.public)[0].id
}
```

`subnet_cidrs` 가 `map(string)` 인 것에 주목하세요. week4는 `map(object({ cidr, az }))` 였습니다. 오늘은 맵의 **key** 가 AZ 이름이고 **값**이 CIDR 입니다. 그래서 `each.key` 가 `availability_zone` 으로, `each.value` 가 `cidr_block` 으로 들어갑니다.

```
resource "aws_subnet" "public" {
  for_each      = var.subnet_cidrs
  availability_zone = each.key
  cidr_block    = each.value
  ...
}
```

[개념] 8. module.network.vpc_id : 내주지 않은 것은 없는 것과 같습니다

모듈의 output 은 `module.<라벨>.<output이름>` 으로 읽습니다. `envs/prod/main.tf` 에서 network 의 값을 compute 로 넘기는 자리가 오늘의 핵심입니다.

```
module "compute" {
  source      = "../../modules/compute"
  vpc_id      = module.network.vpc_id
  subnet_id   = module.network.public_subnet_id
  my_ip       = var.my_ip
  name_prefix = local.name_prefix
  tags        = local.common_tags
}
```

왼쪽은 compute 모듈의 변수 이름이고 오른쪽은 network 모듈의 output 이름입니다. 이 두 줄이 오늘 만드는 유일한 모듈 사이 연결이고, 두 모듈의 결합은 이 파일에서만 일어납니다. `modules/compute` 는 자기가 어느 VPC 에 들어가는지 모른 채 `var.vpc_id` 를 받아 쓸 뿐이고, `modules/network` 는 자기 서브넷에 EC2 가 올라올 것을 모릅니다.

이 연결이 참조이므로 Terraform 이 순서를 스스로 정합니다. `depends_on` 을 적지 않아도 network 가 먼저 만들어지고 compute 가 나중에 만들어집니다. week2에서 본 의존성 그래프가 모듈 사이에도 그대로 적용됩니다.

output 으로 내주지 않은 것은 밖에서 볼 수 없습니다. 예를 들어 network 모듈의 라우트 테이블 ID 를 `envs/prod` 에서 쓰려고 이렇게 적으면 막힙니다.

```
# 이렇게 쓸 수 없습니다
output "rt_id" {
  value = module.network.aws_route_table.public.id
}
```

Error: Unsupported attribute

```
on extra.tf line 2, in output "rt_id":
 2:   value = module.network.aws_route_table.public.id
   |_____
   | module.network is object with 3 attributes
```

This object does not have an attribute named "aws_route_table".

이 오류는 실습워크북 A-5에서 임시 파일 `extra.tf` 로 직접 재현합니다. `module.network is object with 3 attributes` 줄을 보세요. Terraform 입장에서 `module.network` 는 output 세 개(`vpc_id` · `subnet_ids` · `public_subnet_id`)만 가진 객체입니다. 모듈 안에 리소스가 일곱 개 있지만 밖에서는 그 셋만 존재합니다.

모듈 안의 리소스 주소를 밖에서 직접 쓰는 문법은 없습니다. 필요하다면 `outputs.tf` 에 output 을 추가해야 하고, 그것이 계약을 바꾸는 일입니다. 캡슐화가 강제된다는 것이 이 오류의 뜻입니다.



반대 방향도 막혀 있습니다. 모듈 안에서 루트의 `local.common_tags` 나 다른 모듈의 리소스를 볼 수 없습니다. 모듈은 자기가 선언한 변수로 들어온 것만 압니다.

[함정] 9. 모듈이 하나를 골라주면, 고르는 근거가 숨습니다

week4 개념워크북 14번에서 `values(aws_subnet.public)[0]` 을 쓰지 말라고 했습니다. 맵의 정렬이 바뀌면 `[0]` 이 가리키는 대상이 조용히 달라지기 때문입니다. 그런데 오늘 `modules/network/outputs.tf` 에 그 식이 그대로 있습니다.

```
output "public_subnet_id" {
  description = "모듈이 대신 골라 내주는 서브넷. 개념워크북 9번"
  value       = values(aws_subnet.public)[0].id
}
```

모순처럼 보이지만 자리가 다릅니다. week4에서는 리소스가 서브넷을 고르는 자리였고, 오늘은 **모듈이 밖에 내줄 값을 고르는 자리**입니다. 그래서 문제의 모양도 달라집니다.

`values()` 는 맵의 값을 **key 의 사전순**으로 돌려줍니다. 지금 key 는 `ap-northeast-2a` 와 `ap-northeast-2c` 이므로 `[0]` 은 `ap-northeast-2a` 의 서브넷입니다. 여기에 `ap-northeast-2b` 를 더해도 정렬상 가운데로 들어가므로 `[0]` 은 그대로입니다. 이 점은 week4의 경고보다 위험이 좁습니다.

위험이 실제로 드러나는 것은 **사전순 첫 key 를 지우거나 이름을 바꿀 때**입니다. `ap-northeast-2a` 를 맵에서 빼면 `[0]` 이 `ap-northeast-2c` 의 서브넷으로 옮겨갑니다. 그러면 compute 모듈이 받는 `subnet_id` 가 바뀌고, `subnet_id` 는 EC2 를 만든 뒤에는 바꿀 수 없는 속성이라 인스턴스가 교체됩니다. 서브넷 하나를 정리했을 뿐인데 EC2 가 새로 만들어집니다.

더 큰 문제는 그 근거가 어디에도 안 적혀 있다는 것입니다. `envs/prod/main.tf` 만 읽으면 `module.network.public_subnet_id` 라고만 보이고, 어느 AZ 인지는 모듈 안을 열어야 합니다. 모듈이 대신 골라주면서 선택의 근거가 계약 밖으로 숨었습니다.

계약에는 이미 대안이 들어 있습니다. `subnet_ids` output 이 key 가 붙은 맵을 내주고 있으므로, 고르는 일을 부르는 쪽으로 되돌릴 수 있습니다.

```
# 고르는 근거를 envs/prod 로 끌어올린 형태
subnet_id = module.network.subnet_ids["ap-northeast-2a"]
```

이렇게 쓰면 어느 서브넷을 쓰는지 가 조립하는 파일에 그대로 보이고, 맵에서 무엇을 빼든 이 줄이 가리키는 대상은 움직이지 않습니다. 없는 key 를 적으면 그 자리에서 `Invalid index` 로 걸립니다. 실습워크북 B-6에서 심화로 다룹니다.

! 꼭 기억

오늘 실습은 계약을 바꾸지 않고 제공된 `public_subnet_id` 를 그대로 씁니다. 이 항목은 "편의를 위해 모듈이 골라주는 output 을 두면 무엇을 잃는가"를 보라고 넣은 것입니다. 모듈을 설계할 때 되풀이해서 만나는 판단이고, 편한 쪽과 명시적인 쪽 사이의 절충입니다.

Part 4. 주소가 길어집니다

모듈로 옮기면 리소스 자체는 그대로인데 state 에 적히는 주소가 달라집니다. 주소가 달라진다는 것이 week4에서 배운 대로 사고의 씨앗입니다.

[개념] 10. 모듈 경로 접두사

모듈 안 리소스의 주소 앞에는 `module.<라벨>` 이 붙습니다. 라벨은 `module "network"` 의 그 이름입니다.

week4의 주소	오늘의 주소
<code>aws_vpc.main</code>	<code>module.network.aws_vpc.this</code>
<code>aws_subnet.public["a"]</code>	<code>module.network.aws_subnet.public["ap-northeast-2a"]</code>
<code>aws_instance.web</code>	<code>module.compute.aws_instance.web</code>
<code>data.aws_ami.al2023</code>	<code>module.compute.data.aws_ami.al2023</code>

`terraform state list` 가 내주는 10줄은 이렇습니다.

```
module.compute.data.aws_ami.al2023
module.compute.aws_instance.web
module.compute.aws_security_group.web
module.network.aws_internet_gateway.this
module.network.aws_route_table.public
module.network.aws_route_table_association.public["ap-northeast-2a"]
module.network.aws_route_table_association.public["ap-northeast-2c"]
module.network.aws_subnet.public["ap-northeast-2a"]
module.network.aws_subnet.public["ap-northeast-2c"]
module.network.aws_vpc.this
```

주소가 두 군데에서 달라진 것을 보세요. 앞에 `module.network.` 가 붙었고, 서브넷의 key 가 `"a"` 에서 `"ap-northeast-2a"` 로 바뀌었습니다. key 가 바뀐 것은 `subnet_cidrs` 의 맵 구조가 달라졌기 때문이고, 모듈화와는 별개입니다.

셀에서 이 주소를 다룰 때는 week4와 같이 작은따옴표로 감쌉니다. 대괄호와 큰따옴표를 셀이 먼저 해석하기 때문입니다.

```
terraform state show 'module.network.aws_subnet.public["ap-northeast-2a"]'
```

[함정] 11. 모듈을 추가하거나 source 를 바꾸면 init 을 다시 돌립니다

`init` 은 provider 를 받아오는 일만 하는 것이 아닙니다. 자식 모듈이 어디 있는지도 이때 기록합니다. 출력의 첫 두 줄이 그것입니다.

```
Initializing modules...
- network in ../../modules/network
- compute in ../../modules/compute
```

이 정보가 `.terraform/modules` 에 적히므로, `module` 블록을 새로 추가하거나 `source` 경로를 고친 뒤에 `init` 을 다시 돌리지 않으면 Terraform 이 그 모듈을 모릅니다.

```
Error: Module not installed

on main.tf line 1:
1: module "network" {

This module is not yet installed. Run "terraform init" to install all modules
required by this configuration.
```

`plan` 이나 `validate` 가 이 오류를 내면 코드를 의심하지 마세요. `init` 을 안 돌린 것입니다. 모듈 안의 `main.tf` 내용만 고친 경우에는 `init` 을 다시 돌릴 필요가 없습니다. 위치가 바뀐 것이 아니기 때문입니다.

💡 팁

모듈만 다시 받아오는 `terraform get` 이라는 명령도 있습니다. `init` 이 그 일을 함께 하므로 따로 쓸 일은 드뭅니다.

[개념] 12. `moved` 블록: 주소가 바뀔 때 교체를 막습니다

10번의 표처럼 주소가 바뀌면, state 에 이미 리소스가 들어 있는 상태에서는 Terraform 이 그것을 "옛 주소의 리소스를 지우고 새 주소에 만들라"로 읽습니다. 코드를 정리했을 뿐인데 VPC 와 EC2 가 삭제되고 새로 만들어집니다.

`moved` 블록은 주소가 옮겨간 것이라고 알려줍니다.

```
moved {
  from = aws_subnet.public["a"]
  to   = module.network.aws_subnet.public["ap-northeast-2a"]
}
```

모듈 경로 접두사와 인스턴스 key 가 동시에 바뀌어도 한 블록으로 적을 수 있습니다. `moved` 는 state 의 주소만 고치므로 AWS 에는 아무 요청도 보내지 않습니다.

! 꼭 기억

오늘 실습에서는 `moved` 를 쓰지 않습니다. `backend.tf` 의 key 가 `week05/prod/terraform.tfstate` 로 새 값이라서 이 스택의 state 가 비어 있고, 옮길 리소스가 없기 때문입니다. week4 리포의 리소스는 지난주에 이미 destroy 했습니다.

`moved` 가 필요해지는 것은 **살아 있는 인프라를 그대로 두고 코드 구조만 바꿀 때**입니다. 그 상황은 과제의 심화 항목(L3-☆)이고, 6주차의 주제인 state 와 실물의 불일치와 이어집니다.

`moved` 를 쓸 때 주의할 것이 하나 있습니다. 주소만 옮기고 속성이 그대로면 plan 에 아무것도 잡히지 않지만, 옮기면서 태그 같은 속성을 함께 바꾸면 그 부분은 변경으로 잡힙니다. week4의 서브넷 태그는 `${local.name_prefix}-public-${each.key}` 라서 `...-public-a` 였고 오늘은 `${var.name_prefix}-${each.key}` 라서 `...-ap-northeast-2a` 입니다. 이름 태그는 만든 뒤에도 바꿀 수 있는 속성이므로 제자리 수정(~)으로 처리됩니다. 교체되지 않습니다.

Part 5. 환경을 폴더로 가르기

Part 1에서 `dev` 를 만들려면 폴더를 복사해야 한다는 문제로 시작했습니다. 모듈을 뽑은 지금은 무엇이 달라졌는지 확인합니다.

[개념] 13. envs/prod 와 backend 의 key

`envs/prod` 는 리소스를 거의 정의하지 않습니다. `main.tf` 에 있는 것은 `module` 블록 두 개이고, 나머지는 `provider` · 변수 · `locals` · `backend` · `output` 입니다. 조립만 하는 폴더입니다.

환경마다 달라지는 것은 셋입니다.

무엇	prod	dev
backend 의 key	<code>week05/prod/terraform.tfstate</code>	<code>week05/dev/terraform.tfstate</code>
<code>terraform.tfvars</code> 의 값	prod 용 CIDR · 이름	dev 용 CIDR · 이름
<code>local.common_tags</code> 의 Env	<code>prod</code>	<code>dev</code>

모듈 코드는 한 줄도 달라지지 않습니다. week4가 넘긴 "dev 와 prod 를 같은 코드로 만들려면 코드는 같고 `key` 만 달라야 합니다"가 여기서 풀립니다.

`key` 가 환경을 가르키는 이유는 week3에서 본 그대로입니다. 같은 버킷 안에서 `key` 가 다르면 서로 다른 state 이고, state 가 다르면 서로의 리소스를 모릅니다. `envs/dev` 에서 `destroy` 를 해도 prod 의 리소스는 손대지 않습니다.

그리고 `backend` 블록 안에서는 여전히 변수를 쓸 수 없습니다. week4 개념워크북 16번에서 본 제약이 그대로입니다.

Error: Variables not allowed

```
on backend.tf line 3, in terraform:
3:     bucket = var.state_bucket
```

Variables may not be used here.

그래서 `envs/prod/backend.tf` 와 `envs/dev/backend.tf` 는 손으로 적는 파일이 되고, 이것이 환경별로 복사가 남는 유일한 자리입니다. 리소스 정의는 한 벌로 줄였지만 backend 설정은 환경 수만큼 남습니다.

이 마지막 복사마저 없애는 방법이 몇 가지 있습니다. `terraform init -backend-config=...` 로 값을 파일이나 인자로 넘기거나, workspace 를 쓰거나, Terragrunt 같은 도구를 씁니다. 이 스터디에서는 폴더로 가르는 방식만 다룹니다. 무엇이 어디에 적혀 있는지가 눈에 보이는 쪽이 배우기에 낫습니다.

[개념] 14. 같은 모듈, 다른 변수

`envs/dev` 를 추가할 때 하는 일은 `module` 블록에 다른 값을 넘기는 것뿐입니다.

```
# envs/dev/main.tf
module "network" {
  source      = "../../modules/network"
  vpc_cidr    = "10.10.0.0/16"           # prod 는 10.0.0.0/16
  subnet_cidrs = {
    "ap-northeast-2a" = "10.10.1.0/24"
  }
  name_prefix = local.name_prefix
  tags        = local.common_tags
}
```

서브넷을 한 개만 넘겨도 모듈은 그대로 동작합니다. `for_each` 가 맵의 원소 수만큼 도는 것이므로 원소가 하나면 서브넷 하나가 만들어집니다. 모듈을 고칠 필요가 없다는 것이 재사용의 실제 모습입니다.

week4가 넘긴 질문 하나가 남아 있습니다. 리소스 묶음을 여러 벌 만들려면 어디에 `for_each` 를 걸까요. 답은 `module` 블록입니다. `count` 와 `for_each` 는 `resource` 와 `data` 뿐 아니라 `module` 에도 붙는 메타 인자입니다.

```
module "network" {
  source      = "../../modules/network"
  for_each    = var.regions_config
  ...
}
```

이렇게 걸면 주소가 `module.network["seoul"].aws_vpc.this` 처럼 됩니다. 모듈 경로 접두사에 인스턴스 key 가 붙는 것이고, week4에서 본 주소 규칙이 한 층 더 쌓입니다. 오늘 실습에서는 쓰지 않습니다. VPC 를 여러 벌 만들면 계정의 VPC 개수 제한에 걸리고, 실습 목표는 모듈 하나를 정확히 이해하는 것입니다.

Part 6. 비용과 정리

[개념] 15. 이번 주 비용: week4와 같습니다

폴더 구조가 바뀌었을 뿐 만드는 리소스는 week4와 종류도 개수도 같습니다. 그래서 청구서도 같습니다.

항목	개수	시간당	730시간(한 달)
EC2 <code>t3.micro</code> (서울)	1	\$0.0130	\$9.49

항목	개수	시간당	730시간(한 달)
퍼블릭 IPv4 주소	1	\$0.0050	\$3.65
EBS gp3 루트 8GiB	1	\$0.0010	\$0.73
VPC · IGW · 서브넷 2 · 라우트 테이블 · 연결 2 · 시큐리티 그룹	각	\$0	\$0
합계		\$0.0190	\$13.87

week3에서 만든 S3와 DynamoDB는 한 달 \$0.01 미만입니다. 오늘은 그 버킷에 `week05/prod/` 객체 하나를 더 없을 뿐입니다.

이 표의 EBS 8GiB 는 AMI 를 무엇으로 고르는지에 달려 있습니다. `modules/compute/main.tf` 의 이름 필터가 `al2023-ami-2023.*-x86_64` 인 것을 확인하세요. `al2023-ami-*x86_64` 로 느슨하게 두면 루트 볼륨이 30GiB 인 ECS 전용 AMI 가 걸려서 EBS 가 월 \$2.74 가 되고 합계가 약 \$15.88 로 늘어납니다. `plan` 은 그대로 `9 to add` 라서 안전 게이트에 걸리지 않습니다. week3 개념워크북 15번과 실습워크북 B-1 에 같은 경고가 있습니다.

상황	총 비용
3시간 실습하고 바로 destroy	약 \$0.06
destroy 를 잊고 한 달 방치	약 \$13.87

❗ 절대 금지

모듈을 배운 직후에 `module` 블록에 `for_each` 를 걸어 여러 벌 만들어보는 실수가 잦습니다. `compute` 모듈을 두 벌 만들면 EC2 가 2대가 되어 시간당 \$0.0380 이 됩니다. 14번에 왜 오늘 하지 않는지 적어두었습니다.

리팩터링 중에는 EC2 가 잠깐 두 대가 될 수 있습니다. 새 주소로 생성한 뒤 옛 주소를 삭제하는 순서로 계획이 잡히면 그 사이 동안 두 대가 켜집니다. 세션 끝에 `destroy` 확인이 필요한 이유가 늘어났습니다.

NAT Gateway 와 Elastic IP 는 만들지 마세요. `stop` 은 `destroy` 가 아닙니다. 인스턴스를 멈추면 EC2와 퍼블릭 IPv4 요금은 멈추지만 EBS 8GiB의 월 \$0.73은 계속 나갑니다.

[개념] 16. destroy 는 조립하는 쪽에서 한 번만

지우는 것은 `envs/prod` 에서 한 번입니다. 모듈 폴더에서 `destroy` 를 실행할 일은 없습니다. `modules/network` 는 `provider` 도 `backend` 도 없는 폴더이고, 그 안에는 `state` 가 없습니다.

모듈은 코드일 뿐이고, 삭제 대상은 `envs/prod` 가 만든 실제 리소스입니다. 모듈 폴더를 지워도 AWS 의 VPC 는 그대로 남습니다.

```
cd practice/envs/prod
terraform destroy # 두 모듈의 리소스 9개가 전부 지워집니다
```

확인 = 두 겹으로 합니다. `terraform state list` 가 비었다는 것은 Terraform 이 더 이상 관리하지 않는다는 뜻일 뿐입니다. 콘솔에서 손으로 만든 것이나 `destroy` 가 중간에 실패해 남은 것은 여기 안 잡힙니다.

계정 쪽 확인에는 태그를 씁니다. `envs/prod/locals.tf` 가 붙이는 태그는 `Project` · `Study` · `Env` 셋입니다.

무엇	오늘 세션 끝에
<code>Env=prod</code> 태그가 붙은 EC2 · EBS · Elastic IP · NAT Gateway	0개여야 합니다
week3의 S3 버킷(<code>*-tfstate</code>)과 DynamoDB 테이블(<code>*-tflock</code>)	남아 있어야 정상입니다

week3의 state 저장소는 7주차까지 남깁니다. 반대로 하면 state 저장소가 먼저 사라져서 오늘 만든 것을 지울 수단이 없어집니다.

Part 7. 다음 주로 넘기는 문제

[개념] 17. 세 문장으로 줄이면

1. **모듈의 경계는 폴더입니다.** Terraform 은 폴더 하나를 한 덩어리로 읽으므로, 파일을 잘게 쪼개도 경계는 생기지 않습니다.
2. **모듈 사이로 값은 변수와 output 을 통해서만 건너갑니다.** `output` 으로 내주지 않은 것은 밖에서 없는 것과 같고, `locals` 는 폴더 경계를 넘지 않습니다.
3. **모듈로 옮기면 주소에 `module.<라벨>`. 이 붙습니다.** 살아 있는 리소스를 옮길 때는 `moved` 로 알려야 삭제와 생성이 아니라 이름 변경으로 처리됩니다.

그리고 오늘 만든 것들이 다음 주의 문제를 그대로 만들어냅니다.

오늘 본 것	Week6에서 문제가 되는 지점
리소스 주소가 <code>module.network.aws_vpc.this</code> 가 되었습니다	콘솔에서 손으로 만든 리소스는 이 주소를 갖고 있지 않습니다. state 밖의 실물을 코드 안으로 끌어오는 절차가 필요합니다 (<code>import</code>)
<code>moved</code> 는 AWS 에 요청을 보내지 않고 state 의 주소만 고칩니다	state 만 따로 고칠 수 있다는 것은, state 와 실물이 어긋날 수 있다는 뜻입니다 (drift)
<code>plan</code> 이 <code>9 to add</code> 인지를 안전 게이트로 썼습니다	누가 콘솔에서 값을 바꾸면 <code>plan</code> 에 <code>~</code> 가 뜹니다. 코드를 실물에 맞추지 실물을 코드에 맞추지 정해야 합니다
모듈은 계약만 보이고 안은 보이지 않습니다	모듈 안 리소스를 <code>import</code> 하려면 <code>module.network.</code> 까지 붙인 주소를 정확히 적어야 합니다
<code>envs/prod</code> 하나만 만들었습니다	<code>envs/dev</code> 를 추가하면 같은 모듈이 서로 다른 state 두 개에 나뉘어 들어갑니다

참고 문서

- [Modules Overview](#) : Part 1 · Part 2
- [Module Blocks](#) : 4 · 8번
- [Module Sources](#) : 4번. 로컬 경로가 `./` 로 시작해야 하는 이유
- [Providers Within Modules](#) : 5번. provider 상속과 `configuration_aliases`
- [Refactoring](#) : 12번. `moved` 블록
- [values 함수](#) : 9번. 반환 순서가 key 사전순인 근거
- [Module Creation 튜토리얼](#) : 사전 연습 자료
- week2 개념워크북 9 · 12번 : state 주소와 교체(`-/+`)
- week3 개념워크북 4번 : backend 가 변수를 못 받는 이유
- week4 개념워크북 14번 : `values(...)[0]` 을 피하라고 한 원래 맥락
- week4 개념워크북 16번 : backend `key` 로 주차를 가르는 방식

이 문서의 아래 출력은 `solution/` 코드를 Terraform 1.15.8 · AWS provider 6.61.0 · `darwin_arm64` 에서 실제로 실행해 얻은 값입니다. AWS 자격증명 없이 `terraform init -backend=false` 와 `terraform validate` 로 재현할 수 있어서 과금이 없습니다.

출력	어디에
<code>Initializing modules...</code> 와 모듈 두 줄	11번
<code>Error: Unsupported argument</code>	4번
<code>Error: Missing required argument</code>	4번
<code>Error: Invalid module source address</code>	4번의 WARNING
<code>Error: Unsupported attribute</code> (<code>object with 3 attributes</code> 포함)	8번
<code>Error: Module not installed</code>	11번
<code>Error: Variables not allowed</code>	13번

`state list` 10줄의 주소 목록은 코드에서 도출한 것이고, 출력 정렬 순서는 `apply` 로 확인하지 않았습니다. `plan` 의 9 `to add` 와 `destroy` 의 9 `destroyed` 도 리소스 정의를 센 값입니다. 비용은 week4 개념워크북 Part 6에서 조회한 값을 옮겼고, 과금 대상이 EC2 1대 · 퍼블릭 IPv4 1개 · EBS 8GiB 1개로 week4와 같아서 합계가 같습니다. NAT Gateway 요금은 조회하지 않아 숫자를 적지 않았습니다. 원화 환산은 넣지 않았습니다.

다음 → [실습워크북](#) 을 펴고 `practice/envs/prod/` 폴더로 이동하세요.